

Rapport Final PSC INF12

Raisonnement Synergique dans les Systèmes Multi-Agents

Détection synergique d'anomalies réseau par système multi-agents sur graphes de connaissances - application aux réseaux de télécommunications

Étudiants DES BOSCS Raphaël, CHICHE Paul, SIGNORETTI Andrea, QILE Liu
Tuteur TAILHARDAT Lionel, Chercheur IA chez Orange
Coordinateur LUTZEYER Johannes, Professeur assistant, LIX

Table des matières

1	Introduction	3
1.1	Contexte et motivations	3
1.2	État de l'art	3
1.3	Formulation du problème	5
1.4	Objectifs du projet	6
1.5	Principales réalisations	6
1.6	Organisation du rapport	6
2	Résultats intermédiaires et enseignements clés	6
2.1	Pivot stratégique : de la fonction objectif à un système fonctionnel	7
2.2	Apprentissage de JaCaMo par l'implémentation	7
2.3	De cas isolés à des familles alignées sur l'ontologie	7
2.4	Importance de la distinction <i>apriori</i> / <i>aposteriori</i>	8
2.5	Conception des datasets comme instrument scientifique	8
2.6	Enseignements retenus pour l'architecture finale	8
3	Architecture du Système Multi-Agents	9
3.1	Principes de conception	9
3.2	Vue d'ensemble : un pipeline à trois niveaux	9
3.3	Plateforme JaCaMo	10
3.4	L'interface agent-environnement : PythonExecArtifact	11
4	Niveau 1 : Agents de Détection par Facette NORIA-O	12
4.1	Organisation générale : <i>apriori</i> et <i>aposteriori</i>	12
4.2	Famille Structurale	12
4.3	Famille Dynamique	14
4.4	Famille Fonctionnelle	14
4.5	Famille Procédurale	15
5	Niveau 2 : Agents de Raisonnement Conditionnel	16
5.1	Motivation et principe de corrélation	16
5.2	Framework commun <code>common.py</code> et principe d'exécution	16

5.3	Catalogue de diagnoseurs implémentés	17
5.4	Approximation de la fonction objectif : attribution de scores aux diagnostics	19
6	Niveau 3 : Synthèse Narrative par LLM	20
6.1	Séparation décision / communication	20
6.2	Construction du prompt et modèle	21
6.3	Exemple de sortie sur le scénario noria-0.2	21
7	Expériences et Résultats	22
7.1	Catalogue de datasets testé et processus expérimentaux	22
7.2	Premières validations architecturales sur NORIA-O 0.2	23
7.3	Evaluations diverses sur le catalogue synthétique	24
7.4	Etude d'ablation et comparaison avec détecteur et diagnoseur LLM	26
8	Conclusion	29
8.1	Réponse à la problématique	29
8.2	Résultats et réponses empiriques à la problématique	29
8.3	Discussion, limites et perspectives	30
9	Références	31

1 Introduction

1.1 Contexte et motivations

Les réseaux de télécommunications, ou plus largement les infrastructures ICT (*Information and Communication Technologies*), constituent aujourd'hui des systèmes critiques dont dépendent de nombreux services essentiels. Leur supervision soulève une difficulté bien connue : les anomalies y sont rarement localisées dans une seule couche technique. Elles peuvent résulter d'une faiblesse structurelle, d'une évolution logicielle, d'une dépendance fonctionnelle cachée ou d'une rupture procédurale dans la chaîne opérationnelle. La détection et l'interprétation de ces anomalies requièrent donc une capacité à raisonner sur des données hétérogènes, à plusieurs échelles, et avec un fort enjeu d'explicabilité.

Pour traiter cette complexité, Orange s'appuie sur un jumeau numérique représenté sous la forme d'un Graphe de Connaissances. Héritée du Web sémantique, cette représentation permet de relier des sources hétérogènes sous une forme unifiée en triplets RDF. Dans notre contexte, la sémantique du graphe est structurée par l'ontologie **NORIA-O**, qui organise la description du réseau autour de quatre facettes complémentaires : *structurelle*, *dynamique*, *fonctionnelle* et *procédurale*. Cette structuration n'est pas seulement descriptive : elle ouvre la possibilité d'un raisonnement distribué, dans lequel des agents spécialisés interrogent le graphe et produisent des diagnostics partiels, ensuite corrélés à un niveau supérieur.

Le présent PSC s'inscrit dans cette perspective. Il vise à concevoir un système multi-agents capable de détecter et d'interpréter des anomalies dans un graphe de connaissances représentant un réseau de télécommunications, tout en maintenant un haut niveau d'explicabilité. Ce point est central : dans un domaine aussi sensible que l'exploitation réseau, la performance d'un système de diagnostic ne peut être évaluée uniquement à l'aune de son exactitude brute ; elle doit aussi être appréciée en fonction de sa capacité à justifier ses conclusions et à rendre auditable la chaîne de raisonnement qui les produit.

1.2 État de l'art

Notre analyse de l'état de l'art, menée lors de la première phase et consolidée en seconde phase, a confirmé que la combinaison des graphes de connaissances, de l'ontologie NORIA-O et des systèmes multi-agents constitue un cadre théorique pertinent pour maximiser simultanément l'explicabilité et l'efficacité opérationnelle. Elle s'articule autour de quatre axes que nous présentons tour à tour.

1.2.1 Représentation sémantique et graphes de connaissances

La gestion des réseaux TIC fait face à des défis inédits — diversité des services, densité de l'infrastructure et volume de données hétérogènes — qui rendent insuffisante la simple collecte de données brutes. Les opérateurs s'appuient désormais sur des *jumeaux numériques autonomes* [6], mis à jour en temps réel, qui offrent une représentation structurée tout en autorisant l'analyse par des agents.

Le choix des *graphes de connaissances* (KG) [3] s'impose dans ce contexte. Dérivés du Web sémantique, ils reposent sur le modèle *Resource Description Framework* (RDF), dans lequel l'information est structurée en triplets (sujet, prédicat, objet) formant un graphe à arêtes sémantiquement typées. Ce format confère aux KG un haut degré d'*explicabilité*, de *modularité* et de *flexibilité* face à l'hétérogénéité. Appuyée sur les standards du W3C [16], cette représentation autorise l'usage de SPARQL pour interroger les données et identifier des motifs complexes, tandis qu'OWL permet de définir des ontologies et d'en dériver des raisonnements automatiques.

Une *ontologie* est un réseau sémantique définissant les concepts d'un domaine et leurs relations. Dans le domaine TIC, elle peut établir qu'un routeur est un type d'équipement réseau, que deux connexions sont redondantes,

ou qu'un incident critique impacte des services d'urgence. Nous exploitons spécifiquement **NORIA-O** [13], développée chez Orange, qui structure l'information selon quatre facettes — structurelle, fonctionnelle, dynamique et procédurale. NORIA-O se distingue des modèles d'information traditionnels du domaine (TM Forum SID, ITU-T M.3100, TOSCA) par sa sémantique plus riche et son orientation explicite vers la gestion d'incidents. La thèse associée de Tailhardat [12] détaille la construction de ce jumeau numérique exploitable par les agents. En pratique, ces modèles s'exécutent sur des bases de graphes RDF comme *OpenLink Virtuoso* (grands volumes) ou *GraphDB* (inférence automatique) ; notre choix de Virtuoso est discuté en section 3.

Enfin, l'optimisation de tâches sur KG (apprentissage de représentations, raisonnement, complétion) mobilise un ensemble de *fonctions objectif* classiques [15]. L'évaluation de ces fonctions comporte plusieurs dimensions — adaptabilité, effectivité, compatibilité inter-scénario et interprétabilité — ce dernier critère étant central pour nous. Comme détaillé en section 2.1, la formalisation de cette fonction objectif pour piloter l'ordonnancement des agents a été recalibrée en cours de projet : la hiérarchie de sévérité du coordinateur en constitue, dans la présente architecture, une première approximation pragmatique.

1.2.2 Systèmes multi-agents : paradigmes classiques et agentiques

Une fois le jumeau numérique opérationnel, il faut définir les processus qui agissent sur ses données pour le rendre *autonome*. On recourt à des *agents*, entités logicielles capables de percevoir leur environnement, de raisonner et d'agir. En supervision réseau, certains se spécialisent dans la collecte, d'autres dans la détection, d'autres dans la planification d'actions correctives. Cette typologie se double d'un *trade-off explicabilité-précision* : les agents *déterministes* offrent une explicabilité complète au prix d'une complexité élevée, tandis que les *probabilistes* — fondés sur le *process mining* [14] ou sur des plongements de graphes [15] — atteignent une meilleure précision mais perdent en traçabilité. L'article d'Hussain et al. [4] a fourni la base architecturale de notre solution en identifiant les mécanismes de coordination nécessaires ; l'enjeu est alors d'*ordonner* et *prioriser* les agents pour obtenir un compromis entre leurs forces.

Notre implémentation repose sur le paradigme BDI (*Belief- Desire-Intention*) de Rao et Georgeff [9], formalisation canonique d'un agent rationnel qui met à jour ses *croyances* par perception, délibère entre *désirs* (buts candidats) et s'engage sur une *intention* (plan d'action). Ce modèle est implémenté par AgentSpeak/Jason [2] au sein de la plateforme **JaCaMo** [5], qui combine trois composantes complémentaires : *Jason* pour la logique BDI, *CARtAgO* pour les artefacts d'environnement partagés, et *Moise* pour la formalisation de l'organisation (rôles, normes, missions). Cette triple dimension (agents – environnement – organisation) structure notre architecture (section 3).

Nous situons ce choix par rapport à la vague récente des **frameworks agentiques à base de LLM** (AutoGen [17], CrewAI, LangGraph), dans lesquels les agents sont des instances LLM coordonnées par échanges en langage naturel. Ces frameworks offrent une flexibilité remarquable mais présentent deux limites rédhibitoires pour notre cas d'usage : (i) la décision repose sur des inférences LLM probabilistes et non auditables, ce qui est incompatible avec les exigences d'explicabilité du domaine ICT ; (ii) la coordination inter-agents s'appuie sur des prompts en langage naturel plutôt que sur une sémantique formelle, excluant toute garantie sur le comportement du système. Le choix de JaCaMo est donc délibéré : il privilégie la traçabilité formelle du BDI à la flexibilité des échanges LLM, au prix d'une expressivité moindre.

1.2.3 Intégration LLM + graphes de connaissances

L'intégration de modèles de langage et de graphes de connaissances est devenue un champ de recherche à part entière entre 2023 et 2025, dont Pan et al. [8] proposent une taxonomie de référence. Trois paradigmes s'y distinguent. Dans l'approche *KG-enhanced LLM*, le graphe enrichit le contexte du LLM (retrieval-augmented

generation sur KG) pour réduire les hallucinations. Dans l'approche *LLM-augmented KG*, le LLM aide à construire, compléter ou corriger le graphe. Enfin, l'approche *synergized reasoning*, illustrée par Think-on-Graph [11], fait parcourir le graphe au LLM de manière itérative pour raisonner pas à pas.

Notre Niveau 3 se positionne dans un quatrième régime, plus restrictif que les trois précédents : celui du **LLM strictement verbalisateur**. Dans ce régime, le diagnostic est intégralement établi par des composants symboliques en amont (workers L1 + coordinateur L2), et le LLM n'intervient que pour reformuler en prose un rapport structuré déjà complet. Ce choix diffère sensiblement des approches *LLM-raisonneur* : il sacrifie délibérément l'expressivité du LLM en matière de raisonnement pour préserver la traçabilité totale de la décision, propriété indispensable au cadre opérationnel visé. La discussion détaillée de ce choix architectural est menée en section 6.

1.2.4 Explicabilité : le cadre XAI

Notre projet étant centré sur l'explicabilité, nous l'inscrivons dans le champ de l'*IA explicable* (XAI). La taxonomie de référence d'Arrieta et al. [1] distingue deux grandes familles : les modèles *intrinsèquement interprétables* (arbres de décision, règles, modèles linéaires) et les explications *post-hoc* appliquées à des modèles opaques (LIME, SHAP, cartes de saillance). Rudin [10] défend l'idée que, pour les décisions à fort enjeu, les modèles intrinsèquement interprétables doivent être préférés aux explications post-hoc approximatives — thèse directement alignée sur notre choix architectural.

Notre système relève entièrement de la première famille : chaque détection est traçable depuis le triplet RDF initial jusqu'au rapport narratif produit par le LLM. Les requêtes SPARQL des workers L1 sont auditables, les conditions de déclenchement des diagnoseurs L2 sont déclaratives, et le LLM ne reformule que ce que les couches déterministes ont déjà établi. Cette propriété, absente des approches par plongements de graphes où la décision est une boîte noire, est intrinsèque à l'architecture choisie. Le score d'explicabilité que nous définissons en section 5.4 opérationnalise cette intuition : il compte les triplets RDF effectivement traversés pour produire chaque diagnostic, offrant une mesure objective, automatiquement calculable, qui croît avec la richesse du graphe et ne requiert aucune annotation manuelle.

1.3 Formulation du problème

Le problème traité dans ce projet peut être formulé de la manière suivante : *comment concevoir un système multi-agents capable de détecter et de diagnostiquer des anomalies dans un graphe de connaissances représentant un réseau de télécommunications, tout en préservant une forte explicabilité et en maîtrisant la complexité du raisonnement distribué ?*

Cette formulation fait apparaître une tension centrale. D'un côté, la couverture diagnostique exige une spécialisation croissante des mécanismes de détection : plus les agents sont spécialisés, plus ils peuvent capter des motifs fins et localisés. De l'autre, la multiplication de ces agents augmente la difficulté d'orchestration, le risque de redondance, et la complexité de l'interprétation globale. Le problème n'est donc pas simplement de *détecter* des anomalies, mais d'organiser un ensemble de capteurs symboliques de manière à produire un diagnostic cohérent, traçable et exploitable.

Dans notre cadre, cette difficulté se dédouble encore. Il faut d'abord raisonner sur des fragilités *apriori*, visibles sans incident actif, comme l'absence de redondance ou le manque d'information dans le jumeau numérique. Il faut ensuite raisonner *aposteriori* sur des anomalies effectivement survenues, à partir d'événements, de tickets ou de services impactés. L'architecture recherchée doit donc être capable d'articuler plusieurs facettes ontologiques, plusieurs modes d'analyse et plusieurs niveaux de raisonnement.

1.4 Objectifs du projet

Le premier objectif du projet est de proposer une architecture multi-agents fonctionnelle pour la détection d'anomalies sur graphe de connaissances, en s'appuyant sur l'ontologie NORIA-O comme principe de structuration du catalogue de détecteurs. Le deuxième objectif est de garantir l'explicabilité du pipeline diagnostique, en maintenant une chaîne de décision symbolique traçable depuis les requêtes SPARQL de niveau 1 jusqu'aux diagnostics corrélés de niveau 2. Le troisième objectif est de mesurer expérimentalement la pertinence de cette architecture, au moyen d'un dataset de référence réaliste, d'un catalogue synthétique contrôlé et d'un ensemble de KPI adaptés.

À ces trois objectifs principaux s'ajoute un objectif méthodologique : déterminer la place pertinente d'un LLM dans une telle architecture. Plutôt que d'en faire le cœur décisionnel du système, nous avons cherché à évaluer dans quelles conditions il pouvait devenir utile comme couche de synthèse, sans compromettre l'auditabilité des décisions produites en amont.

1.5 Principales réalisations

Le projet a conduit à la réalisation d'un prototype avancé, MASynReas, organisé en trois niveaux. Le niveau 1 regroupe des agents de détection spécialisés, répartis selon les quatre facettes de NORIA-O et selon les deux modes d'analyse *apriori* et *aposteriori*. Le niveau 2 met en œuvre des diagnoseurs corrélatifs, chargés d'agrèger les signaux de niveau 1 autour d'ancres communes et d'attribuer des scores de fiabilité, de sévérité et de priorité. Le niveau 3 est dédié à la synthèse narrative par LLM, dans une logique d'assistance et non de décision.

Sur le plan expérimental, le projet a également produit un catalogue de datasets synthétiques, une instrumentation des KPI, ainsi qu'une série d'expériences comparant le MAS à différentes configurations intégrant un LLM. Ces réalisations donnent au projet une portée qui dépasse le simple prototype logiciel : elles en font aussi une base d'évaluation répliquable. Cette dimension de reproductibilité est renforcée par la disponibilité du code source dans le dépôt GitHub du projet, <https://github.com/Blackbull11/MASynReas>, qui a servi de support commun au développement, à la structuration des familles d'agents, à la génération des datasets et à l'instrumentation des expériences. Dans le cadre d'un travail visant précisément l'explicabilité et l'auditabilité, l'existence d'un tel dépôt n'est pas secondaire : elle participe directement à la traçabilité scientifique du projet.

1.6 Organisation du rapport

Ce rapport est organisé comme suit. La section 2 revient sur les principaux enseignements de la phase intermédiaire du projet et sur les inflexions qui ont progressivement conduit à l'architecture finale. La section 3 présente l'architecture globale de MASynReas : ses principes de conception, le pipeline à trois niveaux, la plateforme JaCaMo et l'artefact `PythonExecArtifact` qui sert d'interface entre agents et graphe de connaissance. Les sections 4, 5 et 6 détaillent chacun des niveaux : respectivement les 54 agents de détection par facette NORIA-O, les 12 diagnoseurs de raisonnement conditionnel fondés sur le mécanisme d'ancrage, et le module de synthèse narrative par LLM. La section 7 rassemble enfin les expériences menées sur `noria-0.2` et sur le catalogue synthétique, afin d'évaluer la complémentarité des familles, la qualité des diagnostics, la robustesse du système et sa comparaison à différentes baselines fondées sur un LLM. La section 8 conclut en synthétisant les contributions, les résultats expérimentaux et les perspectives de travaux futurs.

2 Résultats intermédiaires et enseignements clés

Cette section revient sur les principaux enseignements tirés de la phase intermédiaire du projet. Entre les intentions formulées au lancement du PSC et l'architecture effectivement réalisée, plusieurs inflexions importantes

ont eu lieu. Loin de constituer de simples ajustements contingents, elles ont progressivement clarifié la nature du problème à résoudre, la place respective des agents symboliques et du LLM, ainsi que les conditions nécessaires à une évaluation expérimentale crédible.

2.1 Pivot stratégique : de la fonction objectif à un système fonctionnel

Au début du projet, une part importante de la réflexion portait sur la définition d'une *fonction objectif* globale pour le système multi-agents. L'idée initiale était de caractériser l'utilité collective du MAS en combinant la qualité diagnostique, la gestion de l'urgence et le coût de calcul, puis de déduire de cette expression abstraite une organisation optimale des agents.

En pratique, cette direction s'est rapidement révélée méthodologiquement fragile. Définir une fonction objectif avant de disposer d'une implémentation suffisamment mature revenait à risquer l'optimisation d'un formalisme élégant mais encore déconnecté des capacités réellement implémentables et mesurables. Le projet a donc opéré un pivot *bottom-up*. La priorité n'a plus été de résoudre d'emblée le problème organisationnel dans toute sa généralité, mais de construire d'abord un système de diagnostic fonctionnel, puis d'observer ses propriétés empiriques avant de revenir à des considérations plus abstraites.

Ce pivot s'est révélé scientifiquement fécond. Il a permis de substituer à une ambition purement conceptuelle une base expérimentale beaucoup plus solide : un catalogue explicite de détecteurs et de diagnoseurs, une instrumentation des temps d'exécution, un cadre de KPI, ainsi qu'un catalogue de datasets contrôlés. Dans cette perspective, les trois scores introduits au niveau 2 — fiabilité, sévérité, priorité — peuvent être lus comme une première approximation pragmatique de l'ambition initiale liée à la fonction objectif.

2.2 Apprentissage de JaCaMo par l'implémentation

La phase d'implémentation a également constitué une familiarisation progressive avec la pile multi-agents elle-même. JaCaMo n'a pas été traité comme un simple cadre théorique, mais comme l'environnement d'exécution effectif du projet. Jason a fourni le modèle BDI utilisé pour exprimer les plans, les conditions d'activation et le comportement des contrôleurs. CArAgO a servi de support à l'environnement partagé dans lequel les agents interagissent à travers des artefacts.

Dans le système final, l'artefact central est `PythonExecArtifact`. Son rôle est d'assurer l'interface entre les agents JaCaMo et les scripts Python responsables de l'exécution des requêtes SPARQL et de la production des fichiers JSON. Ce pont Java-Python a d'abord été introduit pour des raisons pragmatiques, mais il est progressivement devenu un atout structurel du projet. Il permet en effet de réserver à JaCaMo la responsabilité de l'orchestration et de la coordination, tout en laissant à des scripts Python légers l'accès au graphe, la sérialisation des résultats et une grande partie de la logique de détection. Cette séparation a fortement réduit le coût d'extension du catalogue d'agents de niveau 1.

2.3 De cas isolés à des familles alignées sur l'ontologie

L'exploration du graphe `noria-0.2` et des premiers prototypes a montré que le diagnostic d'anomalies ne pouvait pas être organisé comme un catalogue monolithique. L'ontologie elle-même suggérait une structuration plus pertinente. Une interface manquante, un burst d'événements, une dépendance de service cachée ou un ticket sans procédure associée ne renvoient pas à la même dimension sémantique du réseau.

L'organisation de NORIA-O en quatre facettes — structurelle, dynamique, fonctionnelle et procédurale — est ainsi devenue un principe de conception central. Cet alignement a eu trois effets bénéfiques. Il a d'abord rendu le catalogue de détecteurs plus lisible sur le plan conceptuel. Il a ensuite amélioré la maintenabilité du code et de la

documentation. Enfin, il a permis d'interpréter les résultats expérimentaux de manière sémantiquement structurée, et non comme une simple liste d'agents déclenchés.

2.4 Importance de la distinction *apriori* / *aposteriori*

Une seconde clarification conceptuelle majeure est apparue après la structuration par familles ontologiques : la distinction entre raisonnement *apriori* et raisonnement *aposteriori*. Le mode *apriori* concerne les fragilités latentes visibles dans le graphe indépendamment d'un incident en cours. Il vise donc des vulnérabilités intrinsèques du réseau ou des manques d'information dans le jumeau numérique : absence d'interface, faible redondance, défaut de couverture fonctionnelle, lacune procédurale ou déficit d'observabilité.

Le mode *aposteriori*, à l'inverse, est centré sur l'anomalie déjà survenue. Il part d'événements, de tickets, de ruptures de service ou d'évolutions du système pour expliquer la cause ou les conséquences opérationnelles de l'incident observé. Cette distinction s'est révélée importante à la fois sur le plan architectural et sur le plan expérimental. Elle permet à une même famille ontologique de supporter deux usages analytiques très différents, et elle explique une partie des écarts observés ensuite dans les benchmarks : les diagnosteurs *apriori* raisonnent souvent par absence, tandis que les diagnosteurs *aposteriori* reposent plus fréquemment sur des évidences positives présentes dans le graphe.

2.5 Conception des datasets comme instrument scientifique

Une autre leçon essentielle de la phase intermédiaire a été que l'évaluation ne pouvait pas reposer exclusivement sur le dataset de référence `noria-0.2`. Bien que ce graphe soit riche et réaliste, il reste un seul cas, avec une distribution particulière des motifs d'anomalie. Or un projet de cette nature exige aussi des scénarios contrôlés dans lesquels les diagnostics attendus sont explicitement connus.

Cette prise de conscience a conduit à la construction d'un véritable cadre de datasets. Deux graphes sains de référence, *Base A* et *Base B*, ont d'abord été définis comme points de départ réutilisables. À partir d'eux, un catalogue de 27 scénarios a ensuite été généré pour couvrir les cas de contrôle, les diagnostics intégrés, les perturbations de robustesse, les situations de classement et les points de scalabilité. Ce catalogue est devenu un instrument expérimental à part entière, indispensable à l'instrumentation des KPI et aux comparaisons systématiques menées dans la section 7.

2.6 Enseignements retenus pour l'architecture finale

La phase intermédiaire a ainsi dégagé plusieurs enseignements qui ont directement façonné l'architecture finale de MASynReas.

1. NORIA-O n'est pas seulement un modèle de données ; c'est aussi une décomposition pertinente de l'espace diagnostique.
2. Le scan déterministe par requêtes SPARQL est particulièrement efficace pour les anomalies formulables comme des configurations locales du graphe, ou comme des absences de relation ou de propriété.
3. Une architecture de diagnostic utile a besoin d'au moins deux niveaux symboliques distincts : un niveau pour détecter, un niveau pour agréger les indices et produire des hypothèses corrélées.
4. Les LLM deviennent utiles lorsqu'ils reçoivent un contexte symbolique structuré, mais ils restent faibles comme scanners exhaustifs du graphe.
5. Une évaluation crédible nécessite à la fois un graphe de référence réaliste et un catalogue de scénarios synthétiques contrôlés.

Ces enseignements servent de transition vers les sections suivantes. La section 3 explicite l'architecture à trois niveaux qui en résulte, puis les sections 4, 5 et 6 détaillent chacune des couches du pipeline.

3 Architecture du Système Multi-Agents

3.1 Principes de conception

L'architecture de MASynReas repose sur quatre principes directeurs, chacun associé à une propriété mesurable du système.

Séparation des responsabilités. Chaque composant du pipeline porte une fonction clairement délimitée. Au niveau 1, un worker correspond à un patron de détection précis ; au niveau 2, un diagnosteur correspond à une famille de corrélation donnée ; au niveau 3, le LLM n'a qu'une fonction de reformulation narrative. Cette séparation réduit fortement le couplage entre modules et garantit qu'une évolution locale — ajout d'un détecteur, ajustement d'un diagnosteur, ou modification du prompt de synthèse — n'introduit pas d'effet de bord dans le reste du système. Elle renforce également l'introspectabilité du MAS : pour tout diagnostic produit, on peut remonter aux détecteurs de niveau 1 impliqués, puis aux motifs SPARQL précis qui ont généré les indices correspondants.

Parallélisme de détection. Les agents de niveau 1 ne communiquent pas entre eux pendant l'exécution. Chacun interroge Virtuoso de manière indépendante et écrit son résultat dans un fichier JSON dédié. Le workspace CArAgO de JaCaMo leur permet de coexister sans se bloquer mutuellement. La conséquence directe est que le temps total de détection est borné par le plus lent des agents, quel que soit leur nombre. Sur noria-0.2, cette propriété procure des accélérations de $12,6\times$ (apriori) et $18,7\times$ (aposteriori) par rapport à une exécution séquentielle équivalente (cf. section 7).

Réactivité BDI asynchrone du coordinateur. Le coordinateur de niveau 2 est un agent BDI (paradigme *Belief-Desire-Intention* implémenté par Jason) qui ne s'active qu'une fois les workers terminés. Il ne consulte pas directement Virtuoso : il lit les fichiers JSON produits par les workers, évalue des conditions logiques inter-familles, et déclenche les diagnosteurs de niveau 2 en conséquence. Cette architecture asynchrone découple complètement le coordinateur de l'ordre d'achèvement des workers.

Diagnostic déterministe avant génération en langage naturel. Le système établit d'abord un diagnostic explicite par raisonnement symbolique, puis seulement dans un second temps le reformule en langage naturel. Autrement dit, la décision précède la communication. Cette contrainte architecturale est essentielle pour préserver un haut niveau d'explicabilité : le LLM n'infère pas lui-même le contenu du diagnostic, il synthétise des résultats déjà établis par les niveaux 1 et 2. On évite ainsi qu'un texte plausible mais insuffisamment justifié soit confondu avec une décision diagnostique auditable.

3.2 Vue d'ensemble : un pipeline à trois niveaux

La hiérarchie du MAS répond à trois fonctions de raisonnement distinctes. Le niveau 1 a pour rôle d'*observer exhaustivement* le graphe sous des angles spécialisés ; le niveau 2 a pour rôle d'*interpréter et de corrélér* les indices produits par le niveau 1 ; le niveau 3 a pour rôle de *communiquer* le diagnostic final sous une forme narrative lisible. Cette hiérarchie n'est donc pas seulement un empilement technique : elle correspond à une décomposition du raisonnement lui-même, depuis la détection locale jusqu'à la restitution à l'opérateur.

Le pipeline s'exécute séquentiellement entre niveaux, mais parallèlement à l'intérieur du niveau 1.

Niveau 1 — détection spécialisée. Le premier niveau regroupe les 54 workers répartis selon les quatre familles NORIA-O (structurale, dynamique, fonctionnelle, procédurale) et selon les deux modes d'exécution (apriori et aposteriori). Chaque worker encapsule une requête SPARQL ou un petit traitement Python associé à un unique

patron d'anomalie. Sa sortie est un fichier JSON autonome, ce qui rend ce niveau à la fois massivement parallélisable et très facile à auditer.

Niveau 2 — corrélation diagnostique. Le second niveau ne repart pas du graphe brut, mais des résultats consolidés de niveau 1. Il active conditionnellement les diagnoseurs pertinents, agrège les indices autour d'ancres communes (ressource, application, service, zone, processus), et produit des diagnostics de plus haut niveau accompagnés de scores de fiabilité, de sévérité et de priorité. Le niveau 2 constitue ainsi le lieu où la synergie du système prend sa forme opérationnelle : plusieurs signaux locaux deviennent un diagnostic unique et interprétable.

Niveau 3 — synthèse narrative. Le troisième niveau reçoit les sorties déjà structurées des niveaux 1 et 2 et les reformule en langage naturel. Son objectif n'est pas de découvrir de nouveaux faits dans le graphe, mais de présenter de manière compacte et intelligible les résultats du raisonnement symbolique. Il s'agit donc d'une couche de communication et non d'une couche de décision.

Concrètement, l'exécution suit le déroulement suivant :

1. Le fichier de configuration `multiagentSystem.jcm` instancie les agents pertinents pour le mode choisi et l'artefact partagé du workspace.
2. L'artefact initialise l'environnement d'exécution, nettoie le dossier `results/` pour garantir un run propre, puis publie le mode actif (*apriori* ou *aposteriori*).
3. Les workers de niveau 1 correspondant à ce mode se déclenchent, lancent leurs scripts Python, interrogent le endpoint Virtuoso et écrivent en parallèle leurs résultats JSON.
4. Une fois ce premier front de calcul achevé, le contrôleur de niveau 2 lit les résultats disponibles et déclenche uniquement les diagnoseurs dont les conditions d'activation sont satisfaites.
5. Les diagnoseurs actifs produisent leurs sorties dans `results/level2/`, avec les ancres diagnostiques et les scores associés.
6. Enfin, le niveau 3 agrège les sorties L1 et L2 en entrée de prompt et produit, lorsqu'il est invoqué, un rapport narratif destiné à la lecture humaine.

Ce pipeline s'exécute en mode *apriori* (analyse préventive sur la structure permanente du graphe, sans contexte d'incident) ou en mode *aposteriori* (analyse contextualisée par des tickets, des incidents ou des signaux d'évolution présents dans le graphe).

3.3 Plateforme JaCaMo

JaCaMo [5] intègre trois composantes complémentaires.

Jason fournit le moteur d'exécution des agents selon le paradigme BDI : chaque agent possède des croyances (*beliefs*), des désirs (*goals*) et des intentions (*plans*). Dans MASynReas, les workers de niveau 1 ont un unique plan : exécuter leur script Python lors de l'initialisation. Le coordinateur de niveau 2 possède une logique plus riche : il attend les signaux de fin de workers, charge les résultats JSON et sélectionne les diagnoseurs à activer.

CARtAgO (*Common ARTifact infrastructure for AGents Open systems*) gère les environnements partagés sous forme d'*artifacts*. L'artefact central de MASynReas est `PythonExecArtifact`, que chaque worker observe (*focus*) et via lequel il invoque son script Python. Cet artefact est le pont entre la logique d'agent (*AgentSpeak*) et l'écosystème Python (requêtes HTTP, calcul numérique, écriture JSON) : les agents JaCaMo ne manipulent jamais directement des sockets ni des fichiers système.

Moise fournit les primitives d'organisation (rôles, missions, normes). Dans la version actuelle, l'organisation est conceptuellement structurée en trois rôles (DéTECTEUR L1, Diagnosticien L2, Narrateur L3) mais sans spécification Moise formelle. Cette formalisation constitue une des perspectives principales du projet (cf. section 8).

3.4 L'interface agent-environnement : PythonExecArtifact

La `PythonExecArtifact` matérialise l'environnement pour les agents du MAS. Cet environnement est un graphe de connaissances RDF structuré par l'ontologie NORIA-O et exposé, dans notre configuration expérimentale, par un endpoint SPARQL Virtuoso local (<http://localhost:8890/sparql>). Chaque run consiste à charger un graphe donné — soit le dataset de référence `noria-0.2`, soit l'un des datasets synthétiques du catalogue — puis à laisser les agents interroger ce même état du monde à travers l'artefact partagé. Les agents ne manipulent donc jamais directement des fichiers Turtle ni des connexions HTTP : toute interaction avec le graphe passe par cette couche d'interface.

La `PythonExecArtifact` expose trois opérations principales.

`execPython(scriptPath)` est l'opération centrale : elle crée un processus fils via `ProcessBuilder`, exécute le script Python désigné et transmet la sortie standard au workspace. Ce *bridge Java-Python* est la clé architecturale du système : il permet aux agents d'exploiter l'intégralité de l'écosystème Python sans que ces dépendances n'entrent dans la couche Jason. Ajouter un nouveau détecteur revient à écrire un script Python et à déclarer un agent dans le JCM, sans aucune recompilation Java.

`execQuery(sparqlQuery)` soumet une requête SPARQL directement via Jena ARQ sans passer par Python. Utilisée pour des requêtes légères ne nécessitant pas de traitement Python.

`writeReport/appendReport` écrivent dans le fichier de rapport consolidé du run, permettant aux agents de laisser une trace narrative sans passer par le LLM.

Dans l'implémentation retenue pour MASynReas, cette interface joue aussi un rôle de normalisation expérimentale : elle impose que tous les agents lisent le même graphe, publient leurs sorties dans une arborescence de résultats commune, et soient exécutés dans un cadre homogène quelle que soit leur famille NORIA-O. Elle constitue donc à la fois un pont technique et une garantie de cohérence entre les différents niveaux du pipeline.

Tous les agents de niveau 1 suivent le même schéma dans le JCM :

Listing 1 – Extrait de `multiagentSystem.jcm` : chaque agent observe le même artefact `w`. Son identité dans le JCM suffit à résoudre le chemin du script Python par convention.

```
mas multiagentSystem {

    agent orphan_resource_detector    { focus: w.pyexec }
    agent missing_interface_detector { focus: w.pyexec }
    // ... (54 agents, meme structure)

    workspace w {
        artifact pyexec : PythonExecArtifact("http://localhost:8890/sparql")
    }
}
```

Cette convention *convention over configuration* élimine tout boilerplate : ajouter le 55^e agent ne demande qu'une ligne dans le JCM.

4 Niveau 1 : Agents de Détection par Facette NORIA-O

4.1 Organisation générale : apriori et aposteriori

Les 54 agents de niveau 1 sont organisés selon deux dimensions orthogonales.

La première est la **famille**, alignée directement sur les quatre facettes de NORIA-O : structurelle, dynamique, fonctionnelle, procédurale. Chaque famille surveille un sous-ensemble des classes et propriétés de l'ontologie qui lui correspond. Cette dimension est toujours explorée dans son ensemble : lorsqu'une famille est activée, tous les agents qu'elle contient pour le mode considéré sont instanciés et exécutés.

La seconde est le **mode d'analyse** :

- **Apriori** : l'agent interroge la structure permanente du graphe sans contexte d'incident. Sa requête SPARQL ne contient aucune référence à un `TroubleTicket` ou `EventRecord` actif. L'objectif est la détection proactive de vulnérabilités, lacunes et incohérences *avant* qu'un incident ne survienne. En pratique, ce mode correspond le plus souvent soit à la détection de fragilités inhérentes au réseau, soit à la mise en évidence d'un manque d'information dans sa représentation sous forme de graphe, c'est-à-dire dans son jumeau numérique.
- **Aposteriori** : l'agent part d'une entité dynamique active (`TroubleTicket` ou `EventRecord`) et traverse le graphe pour expliquer le symptôme observé en le reliant à des causes structurelles, fonctionnelles ou procédurales.

Contrairement à la dimension *famille*, la dimension *mode d'analyse* est sélectionnée explicitement par l'utilisateur dans les réglages du MAS avant l'exécution.

Famille	Apriori	Aposteriori	Total
Structurelle	10	7	17
Dynamique	3	12	15
Fonctionnelle	7	9	16
Procédurale	3	3	6
Total	23	31	54

Table 1 – Distribution des 54 agents de niveau 1 par famille et par mode.

4.2 Famille Structurelle

La famille structurelle surveille la topologie physique du réseau : ressources (`noria:Resource`), interfaces (`noria:NetworkInterface`), liens (`noria:NetworkLink`) et leurs relations de support (`seas:subSystemOf`, `noria:networkInt`). Dans NORIA-O, cet aspect modélise la matérialité du système ICT : les objets techniques, leur connectivité, leur hiérarchie et leur implantation physique.

Agents apriori. Les agents apriori de cette famille recherchent principalement des *fragilités topologiques intrinsèques* et des *angles morts de modélisation* du jumeau numérique.

Pattern détecté	Agent
Ressource structurellement isolée, sans interface, gestion, parent, enfant ni support applicatif	orphan_resource_detector
Ressource sans interface explicite	missing_interface_detector
Interface non rattachée à une ressource	orphan_interface_detector
Interface rattachée mais sans lien réseau	unconnected_interface_detector
Lien réseau avec moins de deux extrémités	incomplete_network_link_detector
Ressource apparemment subordonnée mais sans parent explicite	missing_parent_resource_detector
Application sans ressource de support visible	application_without_support_detector
Ressource sans unité de gestion	unmanaged_resource_detector
Application critique avec support insuffisamment redondant	missing_redundancy_detector
Application critique combinant criticité métier et faiblesse structurelle	criticality_structural_weakness_detector

Table 2 – Catalogue structurel apriori.

Agents aposteriori. Les agents aposteriori contextualisent les faiblesses structurelles autour de ressources ou d'applications déjà impliquées dans un incident.

Pattern détecté	Agent
Ressource incidentée très faiblement connectée	isolated_incident_resource_detector
Incident porté par une ressource connectée à un lien incomplet	incident_on_incomplete_link_detector
Ressource incidentée avec fort impact aval sur ressources ou applications	high_impact_resource_detector
Ressource incidentée sans redondance visible	no_redundancy_incident_detector
Incident impliquant une application au mapping de support incohérent	application_mapping_inconsistency_detector
Concentration spatiale d'incidents sur une même zone	spatial_incident_cluster_detector
Escalade d'incident depuis un composant enfant vers une ressource parente structurante	child_component_incident_escalation_detector

Table 3 – Catalogue structurel aposteriori.

Agent représentatif : `orphan_resource_detector`. Cet agent détecte les ressources n'appartenant à aucune structure opérationnelle. La requête combine cinq `FILTER NOT EXISTS` :

Listing 2 – Requête du détecteur de ressources orphelines. Ce motif de raisonnement par absence est caractéristique des agents apriori.

```

SELECT ?resource WHERE {
  ?resource a noria:Resource .
  FILTER NOT EXISTS {
    ?interface noria:networkInterfaceOf ?resource . }
  FILTER NOT EXISTS { ?resource noria:resourceForApplication ?app . }
  FILTER NOT EXISTS { ?resource noria:resourceManagedBy ?orgUnit . }
  FILTER NOT EXISTS { ?resource seas:subSystemOf ?parent . }
  FILTER NOT EXISTS { ?child seas:subSystemOf ?resource . }
}

```

Ce patron de *raisonnement par absence* — détection de ce qui manque plutôt que de ce qui est présent — est caractéristique des agents apriori. Sur `noria-0.2`, il identifie 14 ressources orphelines.

4.3 Famille Dynamique

La famille dynamique surveille les entités temporelles : événements (`noria:EventRecord`) avec leur horodatage (`noria:loggingTime`) et sévérité (`noria:alarmPerceivedSeverity`), et changements planifiés (`noria:ChangeRequest`). Dans NORIA-O, cet aspect modélise l'évolution du système dans le temps : survenue des événements, changements, escalades, récurrences et successions causales possibles entre faits dynamiques.

Agents apriori. Les agents apriori dynamiques détectent principalement des défauts d'observabilité ou de complétude temporelle dans le graphe.

Pattern détecté	Agent
Demande de changement sans temps effectif complet	<code>change_without_effective_time_detector</code>
Événement sans horodatage	<code>event_without_timestamp_detector</code>
Événement sans élément réseau lié	<code>event_without_related_element_detector</code>

Table 4 – Catalogue dynamique apriori.

Agents aposteriori. Les agents aposteriori dynamiques forment le sous-système de détection temporelle le plus riche du niveau 1. Ils peuvent être regroupés autour de trois grands aspects.

- **Corrélation changement–incident** : l'incident survient peu après un changement, plusieurs changements se chevauchent, ou une dégradation silencieuse suit une modification.
- **Patterns de volume et de simultanéité** : rafales d'événements, répétitions rapprochées, ou incidents touchant plusieurs éléments sur la même fenêtre temporelle.
- **Évolution et propagation de l'incident** : oscillation d'état, ticket rouvert, ticket stale, escalade parent–enfant, absence d'escalade d'un événement critique, ou propagation vers des éléments adjacents.

Pattern détecté	Agent
Incident survenant peu après un changement sur le même élément	<code>change_followed_by_incident_detector</code>
Chevauchement de changements sur un même élément	<code>change_overlap_conflict</code>
Événement critique suivi d'une escalade ticket jugée significative	<code>critical_event_ticket_escalation_detector</code>
Rafale d'événements sur une fenêtre courte	<code>event_burst_detector</code>
Oscillation rapide entre états contradictoires	<code>flapping_state_detector</code>
Propagation courte vers un élément adjacent	<code>incident_propagation_detector</code>
Incidents synchrones sur plusieurs éléments distincts	<code>multi_element_synchronous_incident_detector</code>
Escalade temporelle parent–enfant	<code>parent_child_event_escalation_detector</code>
Réouverture d'incident sur un même élément	<code>reopened_incident_detector</code>
Répétition rapprochée d'événements similaires	<code>repeated_similar_event_detector</code>
Dégradation silencieuse après changement	<code>silent_degradation_after_change_detector</code>
Incident persistant sans résolution visible	<code>stale_incident_detector</code>

Table 5 – Catalogue dynamique aposteriori.

4.4 Famille Fonctionnelle

La famille fonctionnelle surveille les dépendances applicatives : services (`noria:Service`), modules (`noria:ApplicationModule`), applications (`noria:Application`) et leurs relations métier (`noria:resourceForApplication`, `noria:applicationModuleSLA`). Dans NORIA-O, cet aspect modélise la couche de sens métier du réseau : chaînes de support entre ressources techniques, applications, modules et services exposés aux utilisateurs.

Agents apriori. Les agents apriori fonctionnels détectent des lacunes de modélisation et des fragilités de couverture dans la chaîne service–module–application–ressource.

Pattern détecté	Agent
Application sans module fonctionnel	application_without_module_detector
Application sans ressource de support visible	application_without_resource_detector
Ressource mappée à plusieurs applications	duplicated_functional_mapping_detector
Module rattaché à plusieurs services parents	inconsistent_service_hierarchy_detector
Service supporté par trop peu de ressources	over_concentrated_service_detector
Service sans module applicatif rattaché	service_without_application_detector
Service sans couverture de ressources sur toute la chaîne fonctionnelle	service_without_resource_coverage_detector

Table 6 – Catalogue fonctionnel apriori.

Agents aposteriori. Les agents aposteriori fonctionnels raisonnent sur l’impact métier des incidents et sur leur propagation au sein des dépendances applicatives.

Pattern détecté	Agent
Incident applicatif sans ressource de support visible	application_incident_without_resource_detector
Propagation potentielle d’une panne vers plusieurs services	cascading_service_failure_detector
États applicatifs contradictoires au même instant	conflicting_application_state_detector
Dépendance cachée entre services	hidden_service_dependency_detector
Convergence d’incidents au niveau d’un module	module_level_incident_aggregation_detector
Événement ressource sans impact de service traçable	resource_event_without_service_impact_detector
Service impacté par plusieurs ressources distinctes	service_impacted_by_multiple_resources_detector
Service impacté sans escalade ticket associée	service_without_ticket_escalation_detector
Service affecté de façon récurrente	service_with_repeated_incident_detector

Table 7 – Catalogue fonctionnel aposteriori.

4.5 Famille Procédurale

La famille procédurale surveille les processus opérationnels : procédures de remédiation (`noria:OperationPlan`), demandes de changement (`noria:ChangeRequest`) et leurs liaisons avec les tickets et les ressources. Dans NORIA-O, cet aspect modélise la manière dont l’organisation réagit aux anomalies : préparation des procédures, traçabilité entre événements et tickets, et articulation entre incidents et opérations planifiées.

Agents apriori. Les agents apriori procéduraux détectent des signes de préparation opérationnelle incomplète avant même qu’un incident ne soit analysé.

Pattern détecté	Agent
Demande de changement sans créneau planifié	change_request_without_scheduled_time_detector
Procédure sans rattachement visible à un type de ressource	procedure_not_linked_to_resource_type_detector
Ticket sans procédure assignable via les événements liés	ticket_without_assigned_procedure_detector

Table 8 – Catalogue procédural apriori.

Agents aposteriori. Les agents aposteriori procéduraux détectent des ruptures de traçabilité ou des anomalies dans la chaîne opérationnelle active.

Pattern détecté	Agent
Changement lié à plusieurs incidents simultanés	<code>change_linked_to_multiple_incidents_detector</code>
Incident de type événement sans ticket associé	<code>incident_without_ticket_detector</code>
Ticket sans événement lié	<code>ticket_without_linked_event_detector</code>

Table 9 – Catalogue procédural *aposteriori*.

5 Niveau 2 : Agents de Raisonnement Conditionnel

5.1 Motivation et principe de corrélation

Les agents de niveau 2 constituent la couche de *diagnostic déterministe* du système. Alors que le niveau 1 extrait des motifs élémentaires, le niveau 2 vise à transformer ces indices locaux en hypothèses interprétables et hiérarchisables. Sa philosophie est donc la suivante : un diagnostic pertinent n'est pas la simple répétition d'une alerte de niveau 1, mais l'agrégation cohérente de plusieurs signaux issus éventuellement de familles différentes autour d'une même situation opérationnelle.

Cette corrélation repose sur deux notions centrales. La première est celle d'*anchor* : l'entité ou le périmètre commun sur lequel plusieurs résultats de niveau 1 convergent. Selon les cas, cette anchor peut être une ressource, une application, un service, un ticket, une demande de changement, un lien réseau, une localisation ou encore un périmètre procédural. La seconde est celle de *Single Point of Failure* (SPOF), c'est-à-dire un point unique de défaillance : une situation dans laquelle un même composant faible concentre assez d'indices pour faire de lui une cause locale plausible de panne.

Un exemple simple permet d'illustrer ce rôle. `isolated_incident_resource_detector` signale qu'une ressource en incident apparaît faiblement connectée dans la topologie. `no_redundancy_incident_detector` signale que cette même ressource ne dispose d'aucun chemin de secours visible. Pris séparément, ces résultats restent des avertissements locaux. Combinés sur la *même* anchor de type *Resource*, ils fondent un diagnostic plus fort : la ressource constitue vraisemblablement un **Single Point of Failure (SPOF)**. C'est précisément ce saut qualitatif — passer d'un ensemble de résultats atomiques à une interprétation synthétique — qui justifie l'existence du niveau 2.

5.2 Framework commun `common.py` et principe d'exécution

L'orchestration du niveau 2 commence avec l'agent `level2_controller.asl`. Ce contrôleur attend que l'ensemble des détecteurs de niveau 1 correspondant au mode sélectionné aient terminé, puis déclenche uniquement le catalogue de diagnoseurs du même mode. En mode *apriori*, il lance ainsi cinq diagnoseurs ; en mode *aposteriori*, il en lance sept. Dans le premier cas, il arrête automatiquement le MAS une fois ces diagnoseurs terminés.

Du côté Python, tous les diagnoseurs de niveau 2 sont construits sur un framework commun : `src/agt/level2/common.py`. Ce fichier factorise la logique déterministe partagée par tout le catalogue. Chaque diagnoseur y déclare ses entrées, ses règles d'activation, ses anchors possibles, la manière de calculer son diagnostic, et les métadonnées qu'il exporte.

Le framework repose d'abord sur une structure `InputSpec`, qui associe à chaque agent de niveau 1 mobilisé :

- le fichier JSON à lire ;
- la famille NORIA-O d'origine ;
- le mode d'analyse ;
- le rôle de preuve attribué à cet agent ;
- les règles d'extraction des anchors ;
- d'éventuels *tags* servant à enrichir l'évaluation de la sévérité.

Chaque agent de niveau 1 impliqué dans un diagnostic se voit attribuer l'un de trois rôles : **P** pour *Primary evidence*, **S** pour *Supporting evidence*, et **C** pour *Contextual evidence*. Les indices P portent l'évidence directe principale, les indices S renforcent l'hypothèse diagnostique, et les indices C ajoutent du contexte d'impact ou de priorisation. Ces rôles servent à la fois à décider si le diagnosteur doit se déclencher et à pondérer les scores produits ensuite.

Le principe d'exécution commun est le suivant :

1. les fichiers JSON de niveau 1 référencés par le diagnosteur sont chargés ;
2. les enregistrements qu'ils contiennent sont normalisés ;
3. les anchors potentielles sont extraites selon des règles déclaratives ;
4. les résultats sont agrégés par anchor ;
5. une règle d'activation décide si le diagnosteur doit produire au moins une sortie ;
6. une règle de candidature décide, anchor par anchor, si un diagnostic explicite doit être créé ;
7. pour chaque diagnostic retenu, les scores de fiabilité, de sévérité et de priorité sont calculés, puis exportés dans `results/level2/[mode]/[diagnosteur].json`.

Exemple de pipeline : `single_point_of_failure_diagnosteur`. Le diagnosteur `single_point_of_failure_diagnosteur` constitue un bon exemple, car son comportement est simple et fortement interprétable. Il mobilise cinq résultats de niveau 1, tous issus de la famille structurelle *aposteriori* : ressource incidentée isolée, incident sur lien incomplet, ressource à fort impact aval, absence de redondance visible, et escalade depuis un composant enfant. Dans l'implémentation actuelle, quatre de ces entrées sont des preuves primaires et une est une preuve de support. Le diagnosteur s'active dès qu'une anchor concentre au moins deux preuves primaires, puis exporte un diagnostic pour chaque anchor satisfaisant ce même critère.

Le tableau 10 donne l'extrait de matrice de mapping associé à ce diagnosteur.

Agent de niveau 1	Rôle pour <code>single_point_of_failure</code>
<code>isolated_incident_resource_detector</code>	P
<code>incident_on_incomplete_link_detector</code>	P
<code>high_impact_resource_detector</code>	P
<code>no_redundancy_incident_detector</code>	P
<code>child_component_incident_escalation_detector</code>	S

Table 10 – Extrait de la matrice de mapping pour `single_point_of_failure_diagnosteur`.

Cet exemple montre bien le rôle du framework commun : il n'impose pas un diagnostic particulier, mais une *grammaire d'exécution* commune à tout le catalogue. Chaque diagnosteur spécialise ensuite cette grammaire par ses `InputSpec`, ses règles d'activation et son domaine sémantique.

5.3 Catalogue de diagnosteurs implémentés

Le niveau 2 contient actuellement douze diagnosteurs : cinq en mode *apriori* et sept en mode *aposteriori*. Ils sont délibérément **transversaux aux familles NORIA-O**. Autrement dit, ils ne sont pas organisés selon les facettes structurelle, dynamique, fonctionnelle ou procédurale, mais selon des *familles de diagnostic*. Ce choix est essentiel : un diagnostic n'est pertinent que s'il agrège plusieurs patrons élémentaires qui, pris isolément, ne suffisent pas toujours à produire une explication satisfaisante.

Diagnoseurs apriori. Les diagnoseurs apriori cherchent à établir des faiblesses latentes du système avant tout incident explicite.

Diagnoseur	Diagnostic produit	Condition de déclenchement (résumée)
structural_fragility_diagnoser	Diagnosticque une fragilité structurelle sur une ressource ou une chaîne de support accumulant plusieurs faiblesses topologiques.	Se déclenche lorsqu'une même anchor concentre plusieurs signaux structurels primaires ou de support, tels qu'isolation, interface manquante, parentage faible ou redondance absente.
critical_service_exposure_diagnoser	Diagnosticque une exposition critique d'un service ou d'une application métier dont le support technique est trop faible.	S'active lorsque des indices de criticité métier convergent avec des lacunes de couverture ou de redondance sur la même application ou le même service.
observability_gap_diagnoser	Diagnosticque un déficit d'observabilité qui compromettra la reconstruction ultérieure d'un incident.	S'active lorsque des événements ou changements manquent simultanément d'horodatage, d'élément lié ou d'information temporelle exploitable.
procedural_unreadiness_diagnoser	Diagnosticque une impréparation opérationnelle, due à des procédures ou planifications incomplètes.	S'active lorsque des tickets, changements ou procédures présentent ensemble des lacunes de préparation, comme l'absence de procédure assignable ou de créneau planifié.
functional_mapping_gap_diagnoser	Diagnosticque une traçabilité fonctionnelle incomplète ou incohérente entre services, modules, applications et ressources.	S'active lorsque des ruptures ou ambiguïtés de mapping convergent sur la même chaîne fonctionnelle.

Table 11 – Catalogue des diagnoseurs apriori de niveau 2.

Diagnoseurs aposteriori. Les diagnoseurs aposteriori interprètent des incidents, dégradations ou échecs opérationnels déjà présents dans le graphe.

Diagnoseur	Diagnostic produit	Condition de déclenchement (résumée)
single_point_of_failure_diagnoser	Diagnosticque une situation locale où un composant faible constitue un point unique de défaillance.	S'active lorsqu'une même ressource concentre au moins deux preuves primaires de fragilité incidente, telles que l'isolation, l'absence de redondance ou un lien incomplet.
change_induced_incident_diagnoser	Diagnosticque un changement comme cause plausible ou amplificateur d'incident.	S'active lorsqu'un changement, un chevauchement de changements ou une dégradation post-changement convergent sur la même demande de changement ou le même élément lié.
service_cascade_diagnoser	Diagnosticque une propagation montante d'incident à travers une chaîne partagée de support de services.	S'active lorsque plusieurs signaux de propagation fonctionnelle ou dynamique se regroupent autour d'un service, d'un module ou d'une ressource support commune.
traceability_breakdown_diagnoser	Diagnosticque une rupture systématique dans la chaîne de traçabilité incident-événement-ticket-procédure.	S'active lorsque des incidents non ticketés, des tickets sans événement lié, ou des impacts de service sans escalade procédurale se combinent dans un même périmètre opérationnel.
unstable_component_diagnoser	Diagnosticque une instabilité chronique plutôt qu'une panne ponctuelle.	S'active lorsque des indices comme rafales, répétitions, flapping, réouverture ou ancienneté anormale convergent sur un même composant.
application_support_failure_diagnoser	Diagnosticque une défaillance applicative aggravée par une chaîne de support manquante, ambiguë ou contradictoire.	S'active lorsque des incidents applicatifs convergent avec des anomalies de mapping ou d'impact de support sur la même application.
local_infrastructure_cluster_diagnoser	Diagnosticque une zone locale d'infrastructure perturbée plutôt que des pannes indépendantes.	S'active lorsque concentration spatiale, propagation locale, escalade structurelle ou topologie incomplète convergent sur une même localisation ou zone de ressources.

Table 12 – Catalogue des diagnoseurs a posteriori de niveau 2.

5.4 Approximation de la fonction objectif : attribution de scores aux diagnostics

Un diagnostic de niveau 2 n'a pas seulement vocation à exister ou non ; il doit aussi être qualifié. Pour cette raison, chaque sortie de niveau 2 est accompagnée de trois scores distincts : un score de fiabilité, un score de sévérité, et un score de priorité. Cette tripartition répond à un besoin conceptuel clair. Un diagnostic peut être très bien étayé sans être particulièrement grave ; à l'inverse, il peut être potentiellement critique tout en restant appuyé sur une évidence encore limitée. La priorité permet alors d'articuler ces deux dimensions dans une logique de tri opérationnel.

Score de fiabilité. Le score de fiabilité répond à la question : *à quel point les résultats de niveau 1 soutiennent-ils cette hypothèse diagnostique ?* Dans l'implémentation actuelle de `common.py`, il est calculé à partir de la quantité et du type d'évidences présentes autour d'une même anchor. Chaque preuve primaire (P) apporte +20, chaque preuve de support (S) apporte +10. Un bonus inter-familles est ensuite appliqué : +10 si au moins deux familles NORIA-O

sont représentées, +20 si au moins trois le sont. Enfin, un bonus d'ancrage est ajouté : +15 pour les ancors jugées fortes (Resource, Application, Service, ChangeRequest, TroubleTicket, Location) et +5 pour les ancors plus faibles ou plus diffuses. On obtient ainsi :

$$\text{reliability_score} = \min(100, 20 \times \#P + 10 \times \#S + \text{bonus_familles} + \text{bonus_anchor})$$

Le score de fiabilité mesure donc la *force evidencielle* du diagnostic, et non sa gravité opérationnelle.

Score de sévérité. Le score de sévérité répond à une autre question : *si ce diagnostic est vrai, à quel point la situation est-elle grave ?* Dans `common.py`, ce score est obtenu en agrégeant plusieurs composantes :

- une composante de criticité, déduite des champs de type `criticality`, `businessCriticality`, `severity` ou `priority` présents dans les résultats ;
- une composante d'étendue (*breadth*), fondée sur le nombre total de résultats élémentaires agrégés ;
- une composante de concentration, enrichie par les détecteurs porteurs du tag `concentration` ;
- une composante d'aggravation temporelle, alimentée par les détecteurs tagués `temporal` ;
- une composante de faiblesse de gouvernance, alimentée par les détecteurs tagués `governance` ;
- un bonus de base propre à la famille de diagnostic.

De manière schématique, on a donc :

$$\text{severity_score} = \min(100, \text{criticality} + \text{breadth} + \text{concentration} + \text{temporal} + \text{governance} + \text{base_bonus})$$

Ce score exprime la gravité intrinsèque ou systémique du diagnostic, même si son niveau de preuve reste modéré.

Score de priorité. Le score de priorité répond enfin à la question : *quel diagnostic doit être inspecté en premier ?* Dans l'implémentation actuelle, il est calculé par combinaison linéaire des deux scores précédents :

$$\text{priority_score} = \text{round}(0.55 \times \text{reliability_score} + 0.45 \times \text{severity_score})$$

Ce score n'est donc ni un doublon de la fiabilité, ni un simple synonyme de la sévérité. Il joue un rôle de *tri* entre diagnostics concurrents et permet au niveau 2 de produire un ordre d'inspection pertinent sur les scénarios à diagnostics multiples.

Comme exposé en section 2.1, ce score de priorité constitue une *approximation pragmatique de la fonction objectif* initialement envisagée pour le système multi-agents. Nous ne disposons pas encore d'une fonction d'utilité globale pleinement formalisée au sens organisationnel ; en revanche, la priorité synthétise déjà, de manière opérationnelle, deux dimensions essentielles de cette utilité : la confiance dans le diagnostic et le coût potentiel de l'ignorer.

6 Niveau 3 : Synthèse Narrative par LLM

6.1 Séparation décision / communication

Le niveau 3 constitue la couche de *communication* du système. Il reçoit l'intégralité des résultats des niveaux 1 et 2 et produit un rapport en langage naturel à destination des équipes d'exploitation réseau.

La propriété architecturale centrale est que ce niveau **ne prend aucune décision** : le diagnostic est entièrement établi par les niveaux 1 et 2 avant que le LLM ne soit invoqué. Cette séparation *décision / communication* préserve l'explicabilité totale du pipeline : chaque affirmation de la narrative est traçable jusqu'à un résultat JSON de

worker ou de diagnosteur, lui-même traçable jusqu'à un triplet RDF précis du graphe. Cette propriété tranche avec les approches dans lesquelles un LLM est consulté directement pour le diagnostic ; la sous-section consacrée au remplacement du niveau 1 par un LLM détecteur (section 7.4.3) illustre les limites de cette approche (rappel = 0,21 au mieux, raisonnement par absence structurellement impossible).

6.2 Construction du prompt et modèle

Le script `diagnosis_llm.py` agrège les sorties JSON des agents actifs en un prompt structuré en trois blocs :

1. *Level-1 Alerts* : liste des détections L1 actives (type d'anomalie, entité, famille) ;
2. *Level-2 Diagnoses* : diagnosteurs L2 déclenchés, entités anchor et scores de fiabilité, sévérité et priorité ;
3. Instruction de tâche : produire une prose de 3 à 5 phrases, citer les identifiants de ressources exacts, indiquer la sévérité globale, ne pas utiliser de listes à puces.

Le modèle utilisé dans la configuration locale du prototype est **Llama 3.2 :3b** servi localement via **Ollama**. Ce choix répond à deux contraintes opérationnelles : la confidentialité des données réseau (aucune donnée n'est transmise vers l'extérieur) et la compatibilité CPU (3 milliards de paramètres en quantification Q4, inférence en quelques dizaines de secondes sans GPU). Pour les expériences comparatives LLM/MAS des sections 7, **Mistral-7B-Instruct-v0.3** float16 sur GPU (`jaguar.polytechnique.fr`) est utilisé afin d'explorer les capacités d'un modèle plus puissant dans des conditions optimales.

6.3 Exemple de sortie sur le scénario noria-0.2

Appliqué au scénario de référence (défaillance L2 sur RES_TOY_as2, perte d'accès à APP_TOY_PFS01), Llama 3.2 :3b produit :

Listing 3 – Rapport narratif produit par Llama 3.2 :3b sur le scénario noria-0.2. Les identifiants de ressources sont correctement restitués.

Diagnostic Report – Network Incident Analysis

Based on the structural and dynamic evidence collected , the primary incident affects resource RES_TOY_as2, which presents a critical isolation pattern: no redundant network path is available and the associated alarm (SYS_ERROR: L2 table overflow) has propagated to RES_TOY_srv1.

Application APP_TOY_PFS01, which depends on RES_TOY_as2 for its infrastructure , is directly impacted. Trouble ticket TT_TOY2022TT documents this failure but no repair procedure has been assigned.

Recommended action: escalate to the support team responsible for RES_TOY_as2; restore the L2 routing table and verify redundancy configuration before returning the resource to service.

Overall severity: CRITICAL.

La narrative restitue correctement la ressource incriminée (RES_TOY_as2), l'application impactée (APP_TOY_PFS01), le ticket d'incident (TT_TOY2022TT) et l'absence de procédure de remédiation. Ces quatre éléments sont directement

traçables aux sorties JSON des workers `no_redundancy_incident_detector`, `incident_propagation_detector`, `service_without_ticket_escalation_detector` et `ticket_without_assigned_procedure_detector`.

7 Expériences et Résultats

L'ensemble des expériences a été mené à partir d'un endpoint Virtuoso local (`localhost:8890`), avec Python 3.13 et JaCaMo 1.2, en combinant le graphe de référence **noria-0.2** [**noria-dataset**] et le catalogue de datasets synthétiques présenté ci-dessous.

7.1 Catalogue de datasets testé et processus expérimentaux

7.1.1 Deux bases expérimentales complémentaires

Le dispositif expérimental repose sur deux bases de données complémentaires.

La première est le graphe **NORIA-O 0.2** [7], utilisé comme cas de référence réaliste. Il contient **3 152 triplets RDF** et instancie les quatre facettes de l'ontologie NORIA-O [13] :

- **Structurelle** — ressources physiques, interfaces, liens réseau, applications et relations de support ;
- **Dynamique** — événements horodatés et changements planifiés ;
- **Fonctionnelle** — services, modules applicatifs et dépendances de support ;
- **Procédurale** — tickets d'incident et éléments de gestion opérationnelle.

Ce graphe sert essentiellement à valider qualitativement l'architecture sur un cas d'usage crédible, et à observer la manière dont les différentes couches du MAS se comportent sur des données proches du domaine métier.

La seconde base est un **catalogue de 27 datasets synthétiques**, construit à partir de deux graphes sains de référence (Base A et Base B), puis décliné par mutations contrôlées. Chaque scénario est documenté par un graphe `dataset.ttl`, un `manifest.json`, des sorties attendues de niveau 1 et de niveau 2, et un fichier `README.md`. Ces sorties attendues constituent la *ground truth* du banc d'essai : elles ont été définies manuellement lors de la conception de chaque scénario, en construisant le graphe autour d'une anomalie cible précise et en identifiant, à partir des règles formelles des modules concernés, les diagnoseurs qui doivent théoriquement se déclencher. Le catalogue fournit ainsi un véritable banc d'essai pour le MAS.

7.1.2 Structure du catalogue synthétique

Le catalogue a été organisé en familles de scénarios, chacune visant un aspect distinct de la mise à l'épreuve du système.

Famille de scénarios	Objectif expérimental	# datasets
Contrôle et activation	Vérifier le silence du MAS sur des graphes sains ou à évidences faibles, et tester les conditions d'activation	3
Diagnostics intégrés apriori	Exercer les grands diagnoseurs de faiblesse latente du système	7
Diagnostics intégrés aposteriori	Exercer les grands diagnoseurs d'incidents, de propagation et de rupture opérationnelle	8
Robustesse	Tester l'effet de données manquantes, de bruit et de duplications	4
Classement, calibration et scalabilité	Mesurer le tri des diagnostics et l'évolution des temps d'exécution	5

Table 13 – Organisation du catalogue synthétique de 27 datasets.

Cette organisation montre que le catalogue n'a pas été pensé comme une simple collection de graphes jouets, mais comme un dispositif de test couvrant la plupart des propriétés que l'on souhaite établir pour MASynReas : exactitude diagnostique, comportement conservatif, robustesse, priorisation et stabilité temporelle.

7.1.3 Fil directeur des expériences

Les expériences présentées dans cette section répondent à quatre questions complémentaires :

1. **Le cas NORIA-O 0.2 valide-t-il les choix architecturaux de base ?**
2. **Le pipeline L1+L2 est-il exact, robuste et stable sur un benchmark contrôlé ?**
3. **Quelle valeur ajoutée le MAS apporte-t-il à un LLM de synthèse ?**
4. **Un LLM peut-il remplacer le MAS comme détecteur ou comme diagnosteur ?**

La suite de la section suit directement cette progression.

7.2 Premières validations architecturales sur NORIA-O 0.2

Cette sous-section répond à la question 1 du § 7.1.3.

Avant d'exploiter le catalogue synthétique, deux validations sur `noria-0.2` permettent de justifier les choix architecturaux les plus structurants du MAS : d'une part, la décomposition du niveau 1 en agents indépendants, et d'autre part, sa factorisation en quatre familles NORIA-O.

7.2.1 Parallélisme multi-agents

Pour quantifier le gain apporté par l'architecture parallèle de JaCaMo, nous comparons l'exécution multi-agent à une baseline séquentielle qui lance les mêmes scripts Python les uns après les autres. Les deux configurations partagent exactement les mêmes requêtes et le même code de détection ; seule la stratégie d'ordonnancement diffère.

Mode	Agents	Séquentiel	Parallèle (th.)	Accélération
Apriori	23	4 866 ms	386 ms	12,6×
Aposteriori	31	6 514 ms	349 ms	18,7×

Table 14 – Baseline séquentielle vs parallélisme théorique sur `noria-0.2`.

L'architecture multi-agents procure une accélération de **12,6×** en mode apriori et **18,7×** en mode aposteriori. Cette propriété suffit à montrer que le choix d'agents workers indépendants ne répond pas seulement à une logique de modularité : il fournit aussi un avantage de performance concret.

Enseignement. Le niveau 1 parallèle est justifié à la fois comme structure logicielle et comme mécanisme de passage à un grand nombre de détecteurs.

7.2.2 Complémentarité des quatre familles d'agents

La seconde validation qualitative vise à établir que la partition du niveau 1 selon les quatre facettes de NORIA-O n'est pas un simple choix de rangement, mais une propriété utile du système. Il s'agit donc de vérifier que les familles structurelle, dynamique, fonctionnelle et procédurale portent bien des points de vue différents sur un même graphe, et qu'elles préparent effectivement des corrélations intéressantes pour le niveau 2.

Le protocole consiste à exécuter l'ensemble des détecteurs sur `noria-0.2`, puis à agréger les résultats par famille afin d'identifier quelles facettes produisent des signaux, sur quelles entités, et avec quels recouvrements.

Famille	Agents actifs	Détections	Agents totaux	Agent le plus actif
Structurelle	2	17	10	missing_interface (14)
Dynamique	1	10	3	event_without_related_element (10)
Fonctionnelle	2	7	7	inconsistent_service_hierarchy (4)
Procédurale	1	2	3	ticket_without_assigned_procedure (2)
Total	6	36	23	

Table 15 – Résultats des agents de niveau 1 en mode apriori sur noria-0.2.

Mode apriori. Le mode apriori montre une non-redondance nette entre familles. Les faiblesses structurelles, les lacunes d’observabilité, les incohérences fonctionnelles et les carences procédurales ne s’expriment ni sur les mêmes objets ni par les mêmes motifs. Certaines applications, comme APP_TOY_DC01, APP_TOY_DC02 ou APP_TOY_MON01, sont toutefois vues simultanément par plusieurs familles. Ces recouvrements constituent des cibles naturelles pour le niveau 2, qui peut alors transformer plusieurs alertes locales en un diagnostic unique de fragilité ou d’exposition.

Famille	Agents actifs	Détections	Agents totaux
Structurelle	2	10	7
Dynamique	0	0	12
Fonctionnelle	7	19	9
Procédurale	0	0	3

Table 16 – Résultats des agents de niveau 1 en mode aposteriori sur noria-0.2.

Mode aposteriori. En aposteriori, les familles structurelle et fonctionnelle sont actives, tandis que les familles dynamique et procédurale restent silencieuses sur ce cas d’étude. Cette absence de déclenchement ne signale pas une faiblesse de l’architecture, mais une propriété du dataset : noria-0.2 ne contient ni historique temporel assez dense, ni objets `noria:Change`, ni rupture explicite des liens ticket–événement. Dans un graphe de production plus riche, ces deux familles seraient elles aussi sollicitées.

Cette validation confirme que la division du niveau 1 en quatre familles est pertinente. Elle garantit à la fois une spécialisation des capteurs et la possibilité de corrélations inter-familles au niveau 2, qui seraient inaccessibles dans une architecture monolithique.

7.3 Evaluations diverses sur le catalogue synthétique

Cette sous-section répond à la question 2 du § 7.1.3 : le pipeline L1+L2 est-il exact, robuste et stable sur un benchmark contrôlé ?

7.3.1 Métriques retenues

Le catalogue synthétique permet de passer d’une validation qualitative à une évaluation systématique. Les métriques retenues correspondent à des questions différentes sur le comportement du MAS.

Soient TP le nombre de diagnoseurs correctement déclenchés, FP le nombre de diagnoseurs déclenchés à tort, et FN le nombre de diagnoseurs attendus non déclenchés. On définit :

$$\text{Précision}_{L2} = \frac{TP}{TP + FP}, \quad \text{Rappel}_{L2} = \frac{TP}{TP + FN}, \quad F1_{L2} = \frac{2 \cdot \text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Pour les scénarios à diagnostics multiples, nous utilisons en outre le *MRR* (*Mean Reciprocal Rank*) :

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rang}_q}$$

où rang_q est le rang, dans la liste triée par score de priorité, du premier diagnoseur attendu pour le scénario q . Enfin, des mesures de robustesse et de temps d'exécution complètent l'analyse afin d'évaluer respectivement la stabilité du système face aux données dégradées et sa compatibilité avec un usage opérationnel.

7.3.2 Exactitude diagnostique globale

L'expérience principale consiste à exécuter le MAS sur les 27 datasets du catalogue, puis à comparer les diagnoseurs effectivement déclenchés à la ground truth définie pour chaque scénario. Les résultats globaux sont résumés dans le tableau 17.

Mode	Précision _{L2}	Rappel _{L2}	F1 _{L2}
Aposteriori	0,825	0,808	0,815
Apriori	0,177	0,740	0,196
Global	0,394	0,542	0,456

Table 17 – Performance des diagnoseurs de niveau 2 sur le catalogue synthétique. La ligne « Global » est calculée par micro-moyennage (TP, FP, FN cumulés sur toutes les paires mode × dataset) et ne correspond pas à une moyenne des F1 par mode ; elle doit être lue conjointement aux résultats par mode.

Le premier résultat marquant est le très bon comportement du niveau 2 aposteriori, avec un macro-F1 de 0,815. Les diagnoseurs fondés sur des évidences positives présentes dans le graphe — point unique de défaillance, incident induit par changement, cascade de services, rupture de traçabilité — se déclenchent avec une précision élevée et un rappel proche de 0,81. À l'inverse, la performance agrégée en apriori reste limitée. Ce résultat ne signifie pas que le principe de corrélation est inopérant en apriori ; il met plutôt en évidence quelques conditions d'activation encore trop larges, notamment pour `procedural_unreadiness_diagnoser`, qui concentre une part importante des faux positifs.

Enseignement. Le niveau 2 est déjà robuste et pertinent pour les scénarios contextualisés par des incidents ou des tickets. Le chantier restant concerne davantage le resserrement de certains diagnoseurs apriori que l'architecture elle-même.

7.3.3 Classement des diagnostics

Plusieurs scénarios du catalogue ont été conçus pour faire émerger plusieurs diagnostics concurrents sur un même graphe. Dans ce cas, la question expérimentale n'est plus seulement de retrouver les bons diagnoseurs, mais de les ordonner correctement. Le tableau 18 synthétise cette évaluation.

Sous-ensemble	Top-1 Accuracy	MRR
Scénarios multi-diagnostics	1,000	0,833

Table 18 – Qualité du classement sur les scénarios à diagnostics multiples.

Le système retrouve systématiquement le diagnostic principal en première position sur les cas évalués, et place en moyenne les diagnostics attendus à des rangs élevés. Ce point est important : dans une perspective opérationnelle, l'intérêt du niveau 2 n'est pas de produire une simple liste de patterns plausibles, mais de faire émerger rapidement les hypothèses dominantes.

Enseignement. Le score de priorité n'est pas seulement un artefact de présentation ; il joue effectivement son rôle de fonction de triage simplifiée.

7.3.4 Robustesse aux données manquantes et au bruit

Quatre scénarios spécifiques ont été construits pour tester le comportement du système lorsque le graphe est dégradé : suppression de liens structurels ou procéduraux, ajout d'événements hors cible, duplication d'enregistrements.

Dataset	Perturbation	F1 _{L2}
DS19	Données structurelles et temporelles manquantes	1,00
DS20	Liens procéduraux incomplets	1,00
DS21	Événements non pertinents (bruit)	1,00
DS22	Enregistrements dupliqués (bruit)	1,00

Table 19 – Robustesse du MAS sur les scénarios de données dégradées.

Dans les quatre cas, le comportement observé est exactement celui attendu. Les scénarios bruités ne provoquent pas de déclenchements parasites, tandis que les scénarios à information incomplète n'induisent pas de diagnostic abusif. Autrement dit, le système se montre non seulement capable de détecter, mais aussi capable de s'abstenir correctement.

Enseignement. Cette robustesse confirme l'intérêt d'un niveau 1 déterministe et d'un niveau 2 conditionnel : les décisions restent stables lorsque les données sont imparfaites, ce qui est essentiel dans un contexte industriel réel.

7.3.5 Scalabilité

Enfin, trois scénarios de taille croissante ont été exécutés afin d'obtenir une première courbe de passage à l'échelle du pipeline complet.

Dataset	Triplets	L1 (ms)	L2 (ms)	Total (ms)
DS25 (petit)	170	4 505	254	4 759
DS26 (moyen)	258	4 366	242	4 608
DS27 (large)	352	5 823	361	6 434

Table 20 – Temps d'exécution sur trois tailles de graphes synthétiques croissantes.

Sur cette plage de tailles, le temps total de diagnostic reste inférieur à sept secondes. La croissance observée demeure modérée et suggère que, dans notre configuration expérimentale, le coût dominant reste davantage lié à l'orchestration des processus et au démarrage de la JVM qu'au volume des triplets lui-même.

Enseignement. À l'échelle des scénarios considérés, le pipeline complet est compatible avec un usage interactif ou quasi temps réel.

Le catalogue synthétique montre que le MAS ne vaut pas seulement par sa justesse sur quelques cas choisis : il combine exactitude, robustesse et temps de réponse courts sur un ensemble de scénarios construit précisément pour couvrir les principaux risques de test-proofing.

7.4 Etude d'ablation et comparaison avec détecteur et diagnoseur LLM

7.4.1 Principe général des comparaisons avec le LLM

Les expériences suivantes visent à situer la valeur ajoutée du MAS par rapport à une approche plus directement fondée sur un grand modèle de langage. Plutôt que d'opposer abstraitement "MAS" et "LLM", nous considérons trois formes de substitution progressive : laisser le LLM synthétiser directement un rapport à partir du graphe, lui

demander de jouer le rôle d'un détecteur de niveau 1, puis lui demander de classer les diagnostics de niveau 2. Chaque comparaison cible ainsi un rôle précis de l'architecture.

7.4.2 Ablation de la couche MAS pour la synthèse narrative

Cette sous-section répond à la question 3 du § 7.1.3 : quelle valeur ajoutée le MAS apporte-t-il à un LLM de synthèse ?

La première expérience compare trois conditions de synthèse narrative sur cinq scénarios représentatifs : (A) graphe seul, (B) graphe enrichi des sorties L1, et (C) graphe enrichi des sorties L1 et L2. Le modèle utilisé est **Mistral-7B-Instruct-v0.3** en float16 sur GPU, avec trois runs par condition. Les métriques évaluées sont définies comme suit :

- **Conformité** — score dans $\{0, \frac{1}{3}, \frac{2}{3}, 1\}$ égal à la fraction de trois éléments de référence retrouvés dans la réponse : entité anchor attendue, niveau de sévérité attendu, et au moins un mot-clé de diagnostic défini pour le scénario.
- **Hallucination** — proportion de tokens de type identifiant NORIA (chaînes en majuscules avec underscores ou chiffres) présents dans la réponse mais absents du graphe Turtle fourni en entrée.
- **Sévérité** — indicateur binaire (0 ou 1) signalant si le niveau de sévérité attendu (critical, high, etc.) est mentionné dans la réponse.
- **Consistance** — similarité de Jaccard moyenne sur toutes les paires de runs pour un même (scénario, condition), calculée sur les ensembles de mots (tokens lowercasés, dédupliqués).

Condition	Conformité ↑	Hallucination ↓	Sévérité ↑	Consistance ↑
A — graphe seul	0,311	0,000	0,733	0,328
B — graphe + L1	0,533	0,267	0,467	0,433
C — graphe + L1 + L2	0,778	0,133	0,733	0,420

Table 21 – Résultats moyens de l'étude d'ablation GPU (5 scénarios × 3 runs, Mistral-7B-Instruct-v0.3 float16, NVIDIA RTX 4000 Ada, jaguar.polytechnique.fr).

Les résultats font apparaître un gradient clair $A < B < C$ sur la conformité, qui passe de 0,311 à 0,533 puis 0,778. Le point le plus important est toutefois qualitatif : les sorties L1 seules améliorent la conformité, mais introduisent aussi davantage d'hallucinations et une moins bonne calibration de la sévérité. Le niveau 2 joue alors un rôle décisif de structuration : il transforme un ensemble de signaux locaux encore dispersés en hypothèses corrélées, plus faciles à exploiter pour un modèle génératif.

Enseignement. Le MAS n'aide pas le LLM simplement en lui donnant plus d'informations ; il l'aide en lui donnant des informations déjà organisées.

7.4.3 Remplacement du niveau 1 par un LLM détecteur

Cette sous-section aborde la première partie de la question 4 du § 7.1.3 : le LLM peut-il remplacer le MAS comme détecteur ?

La seconde comparaison consiste à demander au LLM d'assumer lui-même le rôle d'un détecteur de niveau 1 à partir du seul graphe `noria-0.2`. Deux stratégies de prompt sont comparées : une stratégie guidée, qui explicite les types d'anomalies à rechercher, et une stratégie ouverte, qui laisse le modèle proposer librement des anomalies.

Mode	Stratégie	Détections LLM	Entités MAS	Rappel	Précision	Temps (s)
Apriori	Guidée	11	33	0,061	0,182	187,7
Apriori	Ouverte	5	33	0,000	0,000	27,8
Aposteriori	Guidée	4	19	0,211	1,000	22,8
Aposteriori	Ouverte	5	19	0,158	0,750	24,4

Table 22 – Performance du LLM seul comme détecteur d'anomalies sur `noria-0.2`.

Les résultats sont sans ambiguïté. Un LLM peut identifier quelques anomalies saillantes, en particulier lorsqu'elles sont appuyées par des indices positifs présents dans le graphe, mais il ne constitue pas un remplaçant crédible des détecteurs déterministes. Le rappel reste très faible, et l'échec est particulièrement marqué en apriori, où les anomalies sont souvent données par absence de relation, d'interface ou de couverture.

Cette baseline montre que le LLM est un mauvais substitut du niveau 1 : sa limite n'est pas seulement quantitative, elle est structurelle. Là où un agent SPARQL vérifie explicitement une contrainte sur l'ensemble du graphe, le LLM demeure un lecteur probabiliste, incapable de garantir l'exhaustivité de son scan.

7.4.4 Remplacement du niveau 2 par un LLM classifieur

Cette sous-section aborde la seconde partie de la question 4 du § 7.1.3 : le LLM peut-il remplacer le MAS comme diagnosteur ?

La dernière comparaison porte sur une tâche plus proche du cœur du niveau 2 : nommer explicitement, parmi les douze diagnosteurs implémentés, ceux qui devraient se déclencher sur un scénario donné. Dans toutes les conditions, la liste des douze noms de diagnosteurs est fournie explicitement dans le prompt : il s'agit donc d'une classification fermée, non d'une génération libre. Trois conditions LLM reprennent le schéma précédent (graphe seul, graphe+L1, graphe+L1+L2) et sont comparées à la référence MAS sur les 36 paires (mode, dataset) du catalogue pour lesquelles au moins un diagnostic est attendu.

Condition	Précision	Rappel	F1	TP / FP / FN
A — LLM, graphe seul	0,111	0,579	0,088	4 / 32 / 20
B — LLM, graphe + L1	0,197	0,639	0,191	7 / 35 / 17
C — LLM, graphe + L1 + L2	0,426	0,685	0,391	9 / 22 / 15
MAS (référence)	0,537	0,778	0,540	13 / 20 / 11

Table 23 – Classification des diagnosteurs L2 : LLM (3 conditions) vs MAS.

Condition	Apriori (16 paires)		Aposteriori (20 paires)	
	F1	TP/FP/FN	F1	TP/FP/FN
A — LLM graphe seul	0,104	2/14/9	0,075	2/18/11
B — LLM graphe + L1	0,148	5/19/6	0,225	2/16/11
C — LLM graphe + L1 + L2	0,244	5/13/6	0,508	4/9/9
MAS (référence)	0,196	5/18/6	0,815	8/2/5

Table 24 – F1 de classification L2 par mode.

Le résultat principal est que le MAS reste globalement supérieur au LLM, même dans sa meilleure condition d'entrée. La F1 globale progresse de 0,088 à 0,191 puis 0,391 à mesure que l'on enrichit l'entrée du modèle, ce qui confirme que les couches du MAS structurent l'information d'une manière utile pour le LLM. Néanmoins, la référence MAS atteint 0,540, avec un avantage très net en aposteriori où elle monte à 0,815 contre 0,508 pour le meilleur LLM. Le léger avantage observé pour le LLM en apriori doit être interprété comme le symptôme d'une faiblesse localisée de certains diagnosteurs du MAS, et non comme une supériorité structurelle de l'approche générative.

L'ensemble des comparaisons avec le LLM converge vers la même conclusion : l'apport principal du MAS n'est pas d'ajouter une couche symbolique décorative autour d'un modèle génératif, mais de fournir l'exhaustivité, la corrélation et la discipline logique qui lui manquent. Le LLM excelle en reformulation et en raisonnement local ; le MAS demeure supérieur dès qu'il s'agit de couvrir systématiquement un graphe et de justifier un déclenchement diagnostique.

8 Conclusion

8.1 Réponse à la problématique

Ce PSC a abordé la question suivante, posée dès la section 1.1 : *comment garantir la synergie d'un système multi-agents dédié à la détection d'anomalies dans les réseaux TIC, de manière à maximiser l'explicabilité tout en contenant la complexité ?*

Pour y répondre, nous avons proposé trois contributions architecturales :

- Une **architecture à trois niveaux explicitement séparés** (détection déterministe L1, raisonnement conditionnel L2, synthèse narrative L3) qui décorrèle les responsabilités : chaque niveau est développé, testé et évalué indépendamment des autres, et la séparation *décision / communication* garantit que le LLM ne prend jamais de décision diagnostique.
- Un **modèle de détection multi-famille orthogonal** : 54 agents organisés en quatre familles NORIA-O (structurelle, dynamique, fonctionnelle, procédurale), chacune spécialisée dans une facette du réseau et chacune opérant en mode apriori ou aposteriori selon le contexte opérationnel.
- Un **framework de corrélation inter-familles** : 12 diagnoseurs de niveau 2 qui agrègent les détections locales des workers en diagnostics consolidés par convergence d'évidence sur une entité anchor, quantifiés par des scores de fiabilité, de sévérité et de priorité.

8.2 Résultats et réponses empiriques à la problématique

L'ensemble du système — 54 agents L1, 12 diagnoseurs L2, 1 narrateur L3 — a été intégré et validé sur le dataset de référence noria-0.2 et un catalogue de 27 datasets synthétiques couvrant des scénarios variés : cas de base, diagnostics multiples, données manquantes, bruit, et montée en charge.

La validation expérimentale apporte cinq réponses concrètes à la problématique.

La synergie est démontrée et quantifiée. Les quatre familles sont non-redondantes sur noria-0.2 (section 7.2.2) : chacune détecte des entités qu'aucune autre ne voit. L'étude d'ablation sur la synthèse narrative (section 7.4.2) montre que passer du LLM seul (condition A) à LLM + L1 + L2 (condition C) améliore la conformité du diagnostic de **+150 %** (0,311 → 0,778) : chaque couche contribue de manière mesurable et indépendante. Sur le catalogue synthétique, le MAS atteint un macro-F1 aposteriori de **0,815** et un MRR de **0,833** sur les cas multi-diagnostics.

L'explicabilité est préservée par construction. Chaque diagnostic est entièrement traçable depuis le rapport narratif LLM jusqu'aux triplets RDF via les fichiers JSON intermédiaires. Les requêtes SPARQL des workers sont lisibles et auditable, les conditions de déclenchement des diagnoseurs L2 sont déclaratives, et le LLM ne reformule que ce que les couches déterministes ont déjà établi. Cette propriété, absente des approches par plongements de graphes (Knowledge Graph Embeddings) où la décision est une boîte noire, est intrinsèque à l'architecture choisie.

La complexité est contenue et le parallélisme est une propriété émergente. L'architecture JaCaMo produit une accélération de 12,6× (apriori) et 18,7× (aposteriori) sans aucune modification des agents individuels (section 7, validation qualitative sur noria-0.2) : le parallélisme émerge de la coexistence des agents dans le même workspace,

sans coordination explicite. Le temps de réponse reste sous 7 secondes sur des graphes de 170 à 352 triplets. L'extensibilité est à coût marginal constant : ajouter le 55^e agent revient à écrire un script Python et une ligne dans le JCM.

Le raisonnement par absence est une limite structurelle des LLMs, pas du MAS. La sous-section consacrée au remplacement du niveau 1 par un LLM détecteur (section 7.4.3) montre qu'un LLM seul atteint un rappel de 0,211 au mieux (aposteriori guidé) et de 0,000 sur les motifs apriori. Les agents SPARQL garantissent un rappel de 1,00 sur leur domaine par construction : l'exhaustivité est structurelle et non probabiliste, ce qui répond directement à la contrainte d'explicabilité du domaine ICT.

La comparaison LLM vs MAS en classification L2 confirme la valeur propre du MAS. Sur 36 paires de test, la sous-section consacrée au remplacement du niveau 2 par un LLM classifieur (section 7.4.4) montre que le MAS atteint $F1 = 0,815$ en mode aposteriori contre $F1 = 0,508$ pour le meilleur LLM (condition C avec contexte L1+L2 complet) : le MAS n'est pas un simple "générateur de contexte" pour le LLM, c'est un meilleur classificateur sur les patterns reposant sur des preuves positives dans le graphe.

8.3 Discussion, limites et perspectives

Les résultats obtenus valident l'hypothèse centrale du projet. La démarche *bottom-up* — construire d'abord un système fonctionnel plutôt que d'optimiser abstraitement une fonction objectif — a produit une base empirique solide : des temps mesurés, des diagnostics validés sur 27 datasets, et une architecture dont les propriétés (parallélisme, non-redondance, robustesse, scalabilité) ont émergé de la conception sans nécessiter d'optimisation a posteriori. Nous avons constaté que la non-redondance observée entre familles NORIA-O est fortement soutenue par la structuration sémantique de l'ontologie elle-même, et non seulement par un choix d'ingénierie supplémentaire.

Ces résultats positifs s'accompagnent de limites identifiées. Premièrement, le diagnosteur `procedural_unreadiness` est surconfiant (15 FP pour 1 TP sur les 16 paires apriori) : ses conditions SPARQL trop larges signalent toute ressource sans plan opérationnel, même dans des graphes propres, abaissant le macro-F1 apriori à 0,196. Deuxièmement, 5 diagnosteurs sur 12 n'ont produit aucun TP sur le catalogue synthétique actuel : leurs cas d'usage ne sont pas encore couverts par les 27 datasets. Troisièmement, le niveau 3 (Llama 3.2 :3b sur CPU) représente 94,5 % du temps end-to-end dans notre configuration locale de référence : l'inférence séquentielle sur CPU est un goulot d'étranglement non adressé dans ce projet. Quatrièmement, l'organisation Moise est restée conceptuelle : les rôles (Détecteur, Diagnosticien, Narrateur) ne sont pas formalisés en spécification Moise, ce qui empêche d'utiliser les normes organisationnelles pour contraindre ou prioriser le comportement des agents.

Ces limites dessinent les perspectives naturelles du travail futur :

- **Affiner les diagnosteurs apriori.** Resserrer les conditions SPARQL de `procedural_unreadiness` (ajouter une contrainte sur la sévérité ou le type de ressource) et calibrer les 5 diagnosteurs sans TP en construisant des datasets couvrant leurs scénarios spécifiques.
- **Agents probabilistes complémentaires.** Intégrer des agents fondés sur des Knowledge Graph Embeddings pour les cas où la preuve est incomplète ou le graphe bruité. Ces agents cohabiteraient avec les workers déterministes : le framework de corrélation L2 est agnostique au type d'agent L1.
- **Formalisation Moise et fonction objectif.** Définir les rôles, missions et normes du système via une spécification Moise explicite, puis implémenter la fonction objectif (envisagée dès septembre 2025) comme une métrique d'utilité organisationnelle pilotant la priorisation des diagnosteurs actifs.
- **Optimisation du niveau 3.** Servir le LLM sur GPU dédié (`jaguar.polytechnique.fr` avec Mistral-7B-Instruct-v0.3 float16 est déjà disponible et validé sur les comparaisons de la section 7.4), ou déporter la narrative en tâche asynchrone pour ne pas bloquer le diagnostic.
- **Enrichissement dynamique du graphe.** Systématiser la réinjection des diagnostics inférés dans le graphe

afin que les agents futurs bénéficient de l'historique des diagnostics passés : un graphe de connaissances dynamique et auto-annoté.

Références

9 Références

- [1] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser et al. « Explainable Artificial Intelligence (XAI) : Concepts, taxonomies, opportunities and challenges toward responsible AI ». In : *Information Fusion* 58 (2020), p. 82-115. doi : 10.1016/j.inffus.2019.12.012.
- [2] R. H. Bordini, J. F. Hübner et M. Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason*. Wiley Series in Agent Technology. John Wiley & Sons, 2007. isbn : 978-0-470-02900-8.
- [3] A. Hogan, E. Blomqvist, M. Cochez et al. *Knowledge Graphs*. Synthesis Lectures on Data, Semantics, and Knowledge. Springer, 2021. doi : 10.1007/978-3-031-01918-0.
- [4] A. Hussain, A. A. Bui et Y. K. Suh. « Multi-agent systems and their applications ». In : *International Journal of Computing and Digital Systems* 6.3 (2017), p. 133-140.
- [5] JaCaMo Project. *JaCaMo : A Multi-Agent Oriented Programming Platform*. <https://jacamo-lang.github.io/jacamo/>. Consulté en avril 2026. 2024.
- [6] M. Z. Muchika. « Knowledge Management Framework for Autonomous Digital Twins ». Thèse de doct. Aalto University, 2023.
- [7] Orange OpenSource. *NORIA Ontology Dataset (noria-0.2)*. GitHub repository. 2023. url : <https://github.com/Orange-OpenSource/noria-ontology/tree/master/dataset/noria-0.2>.
- [8] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang et X. Wu. « Unifying Large Language Models and Knowledge Graphs : A Roadmap ». In : *IEEE Transactions on Knowledge and Data Engineering* 36.7 (2024), p. 3580-3599. doi : 10.1109/TKDE.2024.3352100.
- [9] A. S. Rao et M. P. Georgeff. « BDI Agents : From Theory to Practice ». In : *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*. San Francisco, CA, USA, 1995, p. 312-319.
- [10] C. Rudin. « Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead ». In : *Nature Machine Intelligence* 1.5 (2019), p. 206-215. doi : 10.1038/s42256-019-0048-x.
- [11] J. Sun, C. Xu, L. Tang et al. « Think-on-Graph : Deep and Responsible Reasoning of Large Language Model on Knowledge Graph ». In : *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. 2024. url : <https://openreview.net/forum?id=nnV01PvbTv>.
- [12] L. Tailhardat. « Anomaly Detection using Knowledge Graphs and Synergistic Reasoning : Application to Network Management and Cyber Security ». Thèse de doctorat. Sorbonne Université, 2024. url : <https://thesjaces.hal.science/THESES-SU/tel-04831512v1>.
- [13] L. Tailhardat, Y. Chabot et R. Troncy. « NORIA-O : An Ontology for Anomaly Detection and Incident Management in ICT Systems ». In : *Semantic Web Journal* (2023). url : <https://www.semantic-web-journal.net/system/files/swj3334.pdf>.
- [14] W. van der Aalst. *Process Mining : Data Science in Action*. 2^e éd. Springer, 2016. doi : 10.1007/978-3-662-49851-4.

- [15] Q. Wang, Z. Mao, B. Wang et L. Guo. « Knowledge Graph Embedding : A Survey of Approaches and Applications ». In : *IEEE Transactions on Knowledge and Data Engineering* 29.12 (2017), p. 2724-2743. doi : 10.1109/TKDE.2017.2754499.
- [16] World Wide Web Consortium. *Semantic Web Standards*. <https://www.w3.org/standards/semanticweb/>. Consulté en avril 2026. 2023.
- [17] Q. Wu, G. Bansal, J. Zhang et al. *AutoGen : Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. arXiv : 2308.08155 [cs.AI]. url : <https://arxiv.org/abs/2308.08155>.